Demo by Yeana Bond on 4th, March 2026

FLIPPED CLASS: SINGLY CONNECTED LINKED LISTS

PAIRED ACTIVITY
QUIZ
DISCUSSION
HEADS-UP ON THE GROUP ASSIGNMENT

DATA STRUCTURES & ALGORITHMS

A singly connected linked list

- a fundamental linear data structure where elements, called **nodes**, are connected in a sequence using **pointers**
- **Node**: The basic building block of a linked list, consisting of a data field and a "next" pointer.
- **Head**: A pointer to the first node in the list. It is the starting point for all operations and traversal.
- **Tail**: The last node in the list. Its "next" pointer is always **NULL**, signifying the end of the list.
- **Traversal**: The process of visiting each node in the list, starting from the head and following the "next" pointers until the end.
- **Unidirectional**: Data can only be traversed in one direction (forward), like a train that can only move from one coach to the next.

DATA STRUCTURES & ALGORITHMS

A singly connected linked list's common operations

- **Insertion**: Adding a new node at the beginning, at the end, or after a specific node.
- **Deletion**: Removing a node from the beginning, end, or a specific position.
- **Traversal**: Iterating through the entire list to perform an action on each node.
- **Searching**: Finding a node with a specific value (O(n) time complexity).
- **Reversal**: Changing the order of the nodes in the list

Emergency Room check-in
prototype code review via visualization, go to:

HTTPS://GITHUB.COM/YEANAVT/EMERGENCYROOMCHECKIN

ER Waiting Room — Linked List Visualizer

Singly-linked list sorted by pain level (descending)

head node | newly inserted | existing node

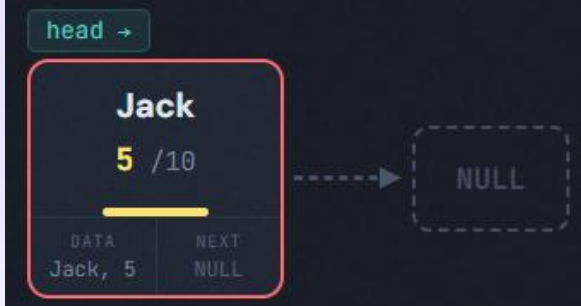
Initial | Jack (5) | Jose (6) | Chris (10) | Mary (6) | Jay (2)

◀ Prev | Next ▶

Step 1 of 5

```
waiting_room->check_in("Jack", 5);
```

Case 2: Empty list. Jack with pain **5** becomes the head. `newnode->next = NULL`.



$ Jack (5) → End of waiting room list for now!

ER Waiting Room — Pointer Trace Visualizer

Watch compare & compare2 traverse the singly-linked list

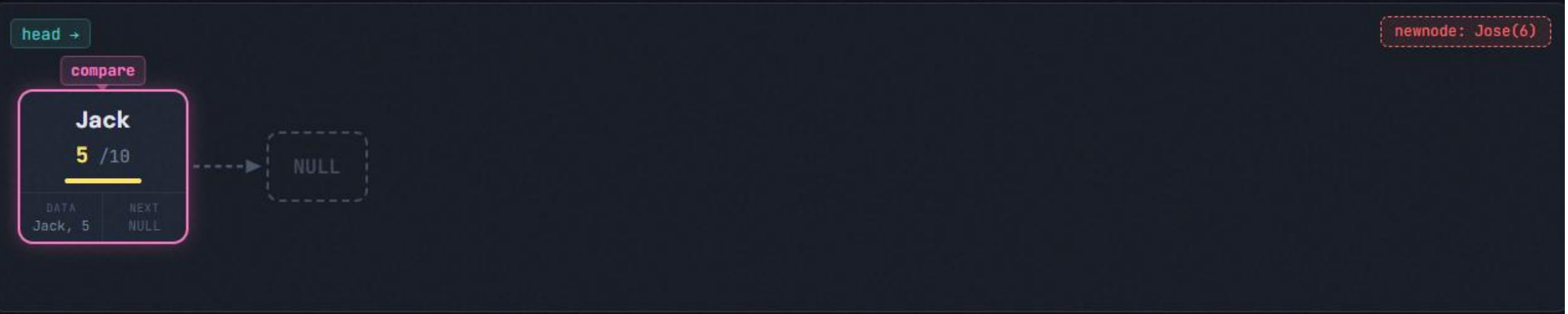
head | new node | compare | compare2 | existing

Init | Jack (5) | Jose (6) | Chris (10) | Mary (6) | Jay (2)

◀ Prev Sub-step 1 / 3 Next ▶

```
waiting_room→check_in("Jose", 6);
```

New node: **Jose(6)**. head != NULL → enter Case 3. Set **compare = head** (Jack).




```
$ Jose 6
```

ER Waiting Room — Pointer Trace Visualizer

Watch compare & compare2 traverse the singly-linked list

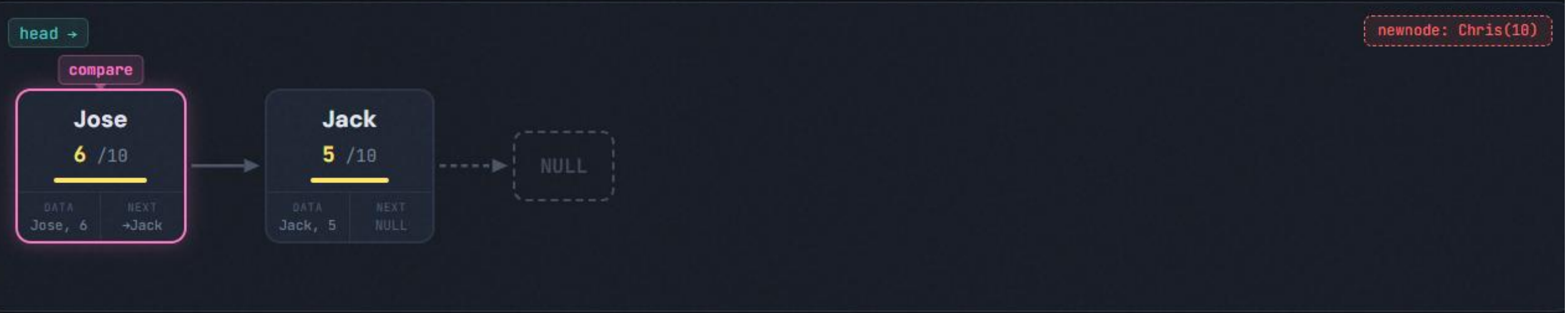
head | new node | compare | compare2 | existing

Init | Jack (5) | Jose (6) | Chris (10) | Mary (6) | Jay (2)

◀ Prev Sub-step 1 / 3 Next ▶

```
waiting_room->check_in("Chris", 10);
```

New node: **Chris(10)**. head != NULL → Case 3. Set **compare = head** (Jose).




```
$ Chris 10
```

ER Waiting Room — Pointer Trace Visualizer

Watch compare & compare2 traverse the singly-linked list

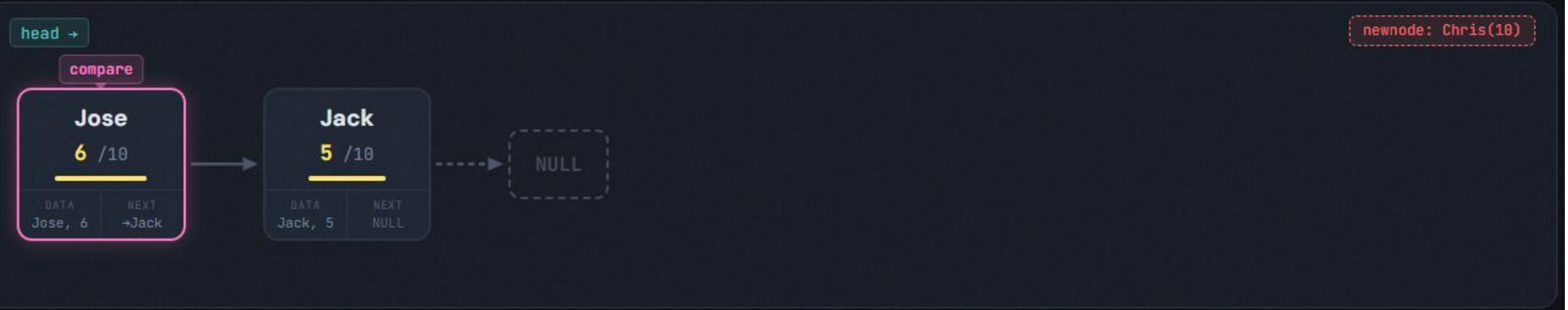
head | new node | compare | compare2 | existing

Init | Jack (5) | Jose (6) | Chris (10) | Mary (6) | Jay (2)

◀ Prev Sub-step 2 / 3 Next ▶

```
if (pain > compare→pain) // 10 > 6? → TRUE
```

Pain **10** > **compare**→pain **6**? YES → **Sub-case 3a: insert at front.** No traversal needed.




```
$ Chris 10
```

ER Waiting Room — Pointer Trace Visualizer

Watch compare & compare2 traverse the singly-linked list

head | new node | compare | compare2 | existing

Init | Jack (5) | Jose (6) | Chris (10) | Mary (6) | Jay (2)

◀ Prev Sub-step 2 / 9 Next ▶

```
if (pain > compare→pain) // 6 > 10? → FALSE
```

Pain 6 > compare→pain 10? NO → enter else block. Initialize compare2 = NULL. Enter while loop.

```
$ Mary 6
```

ER Waiting Room — Pointer Trace Visualizer

Watch compare & compare2 traverse the singly-linked list

head | new node | compare | compare2 | existing

Init | Jack (5) | Jose (6) | Chris (10) | Mary (6) | Jay (2)

◀ Prev Sub-step 3 / 9 Next ▶

```
while (pain ≤ compare→pain && compare→next ≠ NULL) // 6 ≤ 10 && next≠NULL? → TRUE
```

While condition: 6 ≤ 10? YES. Chris→next ≠ NULL? YES → enter loop body.

$ Mary 6

ER Waiting Room — Pointer Trace Visualizer

Watch compare & compare2 traverse the singly-linked list

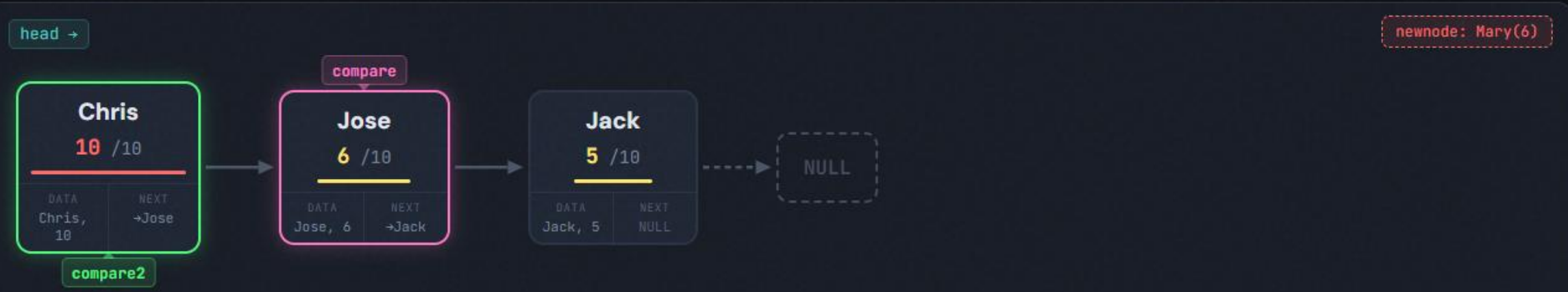
head · new node · compare · compare2 · existing

Init | Jack (5) | Jose (6) | Chris (10) | Mary (6) | Jay (2)

◀ Prev Sub-step 4 / 9 Next ▶

```
compare2 = compare; // compare2 = Chris
compare = compare→next; // compare = Jose
```

compare2 catches up to Chris(10). **compare** advances to Jose(6). Pointers moved one step right.




```
$ Mary 6
```

ER Waiting Room — Pointer Trace Visualizer

Watch compare & compare2 traverse the singly-linked list

head | new node | compare | compare2 | existing

Init | Jack (5) | Jose (6) | Chris (10) | Mary (6) | Jay (2)

◀ Prev Sub-step 5 / 9 Next ▶

```
while (pain ≤ compare→pain && compare→next ≠ NULL) // 6 ≤ 6 && next≠NULL? → TRUE
```

While condition: 6 ≤ 6? YES. Jose→next ≠ NULL? YES → loop body again.

```
$ Mary 6
```

ER Waiting Room — Pointer Trace Visualizer

Watch compare & compare2 traverse the singly-linked list

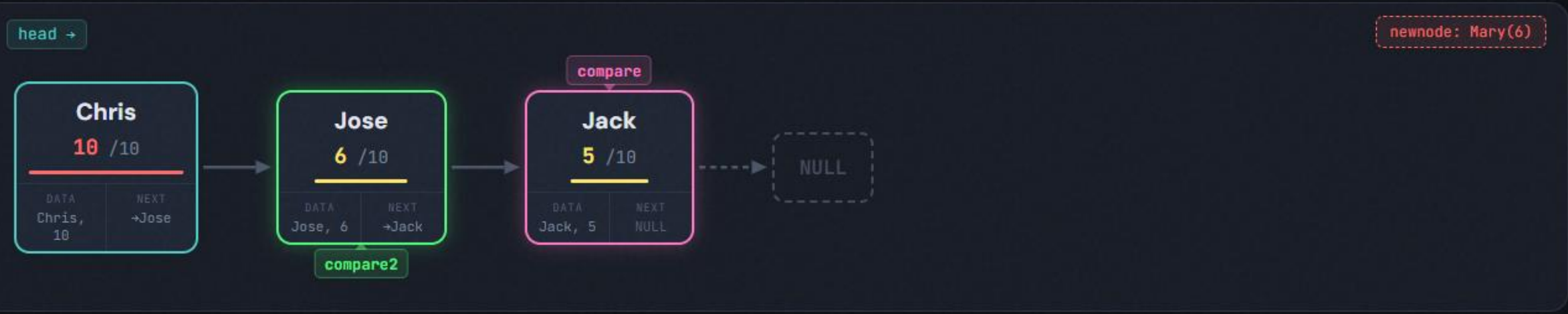
head | new node | compare | compare2 | existing

Init | Jack (5) | Jose (6) | Chris (10) | Mary (6) | Jay (2)

◀ Prev Sub-step 6 / 9 Next ▶

```
compare2 = compare;  // compare2 = Jose
compare = compare→next;  // compare = Jack
```

compare2 advances to Jose(6). **compare** advances to Jack(5). Both pointers shifted one more step.




```
$ Mary 6
```

ER Waiting Room — Pointer Trace Visualizer

Watch compare & compare2 traverse the singly-linked list

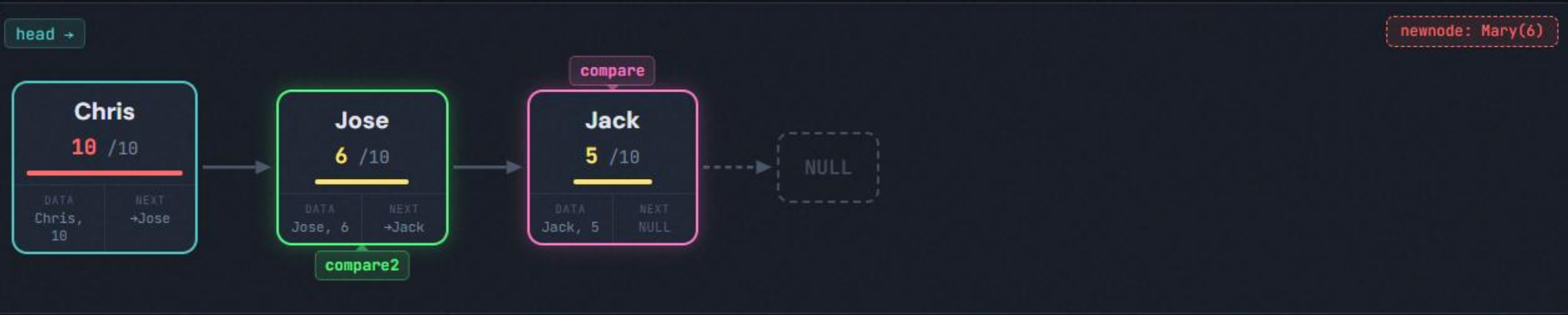
head | new node | compare | compare2 | existing

Init | Jack (5) | Jose (6) | Chris (10) | Mary (6) | Jay (2)

◀ Prev Sub-step 7 / 9 Next ▶

```
while (pain ≤ compare→pain && ...) // 6 ≤ 5? → FALSE — exit while
```

While condition: 6 ≤ 5? NO → exit while loop. compare = Jack(5), compare2 = Jose(6).




```
$ Mary 6
```

ER Waiting Room — Pointer Trace Visualizer

Watch compare & compare2 traverse the singly-linked list

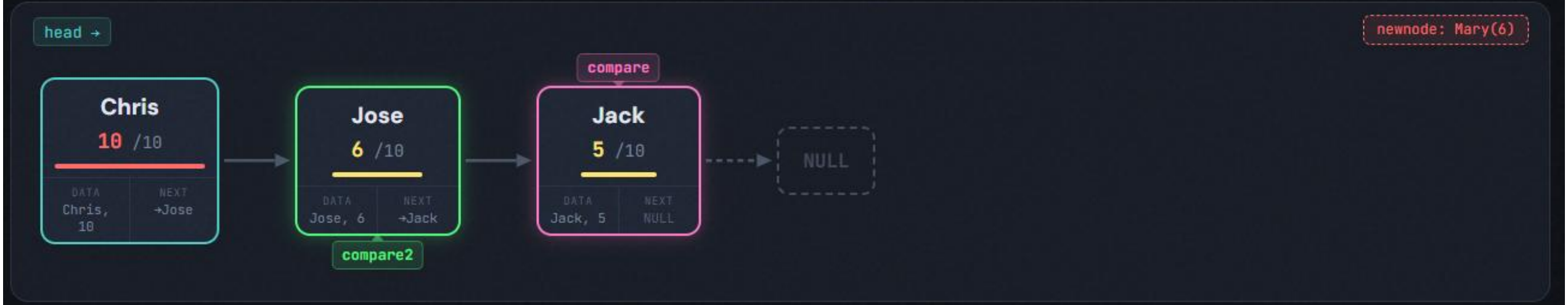
head | new node | compare | compare2 | existing

Init | Jack (5) | Jose (6) | Chris (10) | Mary (6) | Jay (2)

◀ Prev Sub-step 8 / 9 Next ▶

```
if (pain > compare→pain) // 6 > 5? → TRUE → Sub-case 3b-i
```

6 > compare→pain 5? YES → Insert Mary between compare2 (Jose) and compare (Jack).




```
$ Mary 6
```

ER Waiting Room — Pointer Trace Visualizer

Watch compare & compare2 traverse the singly-linked list

head · new node · compare · compare2 · existing

Init | Jack (5) | Jose (6) | Chris (10) | Mary (6) | Jay (2)

◀ Prev Sub-step 9 / 9 Next ▶

```
compare2->next = newnode;  // Jose→Mary
newnode->next = compare;   // Mary→Jack
```

Inserted! Mary(6) is spliced between Jose and Jack. FIFO among equal pain levels — Jose(6) stays ahead of Mary(6).

```
$ Mary 6
$ Chris (10) → Jose (6) → Mary (6) → Jack (5) → End of waiting room list for now!
```

ER Waiting Room — Pointer Trace Visualizer

Watch compare & compare2 traverse the singly-linked list

head | new node | compare | compare2 | existing

Init | Jack (5) | Jose (6) | Chris (10) | Mary (6) | Jay (2)

◀ Prev Sub-step 3 / 12 Next ▶

```
while (...) // 2 ≤ 10 && next≠NULL? → TRUE
```

2 ≤ 10 and Chris→next exists → enter loop.

```
$ Jay 2
```

ER Waiting Room — Pointer Trace Visualizer

Watch compare & compare2 traverse the singly-linked list

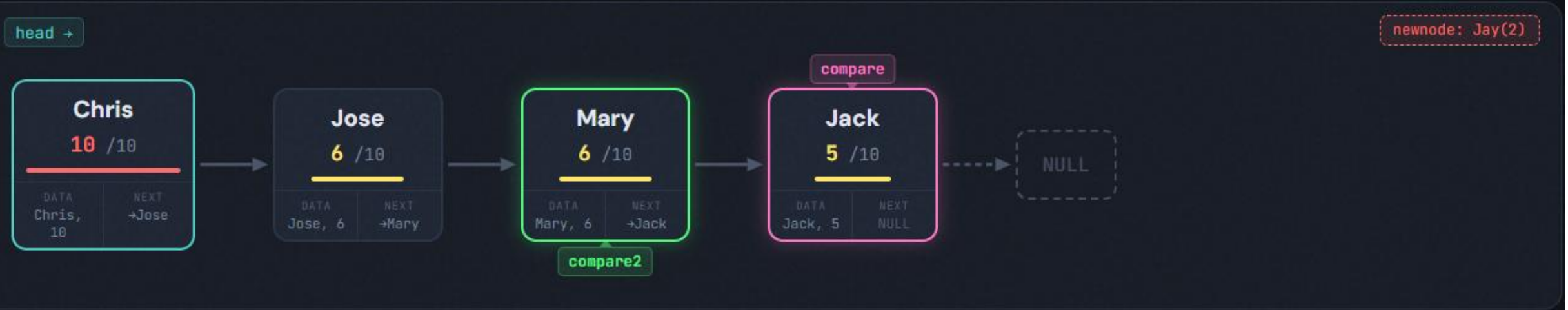
head | new node | compare | compare2 | existing

Init | Jack (5) | Jose (6) | Chris (10) | Mary (6) | Jay (2)

◀ Prev Sub-step 9 / 12 Next ▶

```
while (...) // 2 ≤ 5 && next≠NULL? → FALSE (Jack→next == NULL)
```

2 ≤ 5? YES, but Jack→next == **NULL** → **exit while loop.** We've reached the tail.




```
$ Jay 2
```

ER Waiting Room — Pointer Trace Visualizer

Watch compare & compare2 traverse the singly-linked list

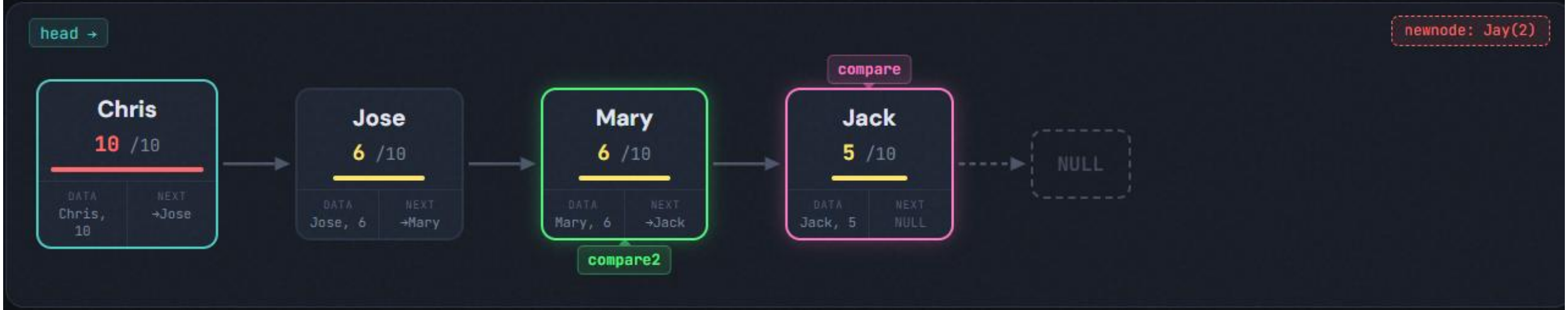
head | new node | compare | compare2 | existing

Init | Jack (5) | Jose (6) | Chris (10) | Mary (6) | Jay (2)

◀ Prev | Sub-step 10 / 12 | Next ▶

```
if (pain > compare→pain) // 2 > 5? → FALSE
```

2 > 5? NO → enter outer else block.



$ Jay 2

ER Waiting Room — Pointer Trace Visualizer

Watch compare & compare2 traverse the singly-linked list

head | new node | compare | compare2 | existing

Init | Jack (5) | Jose (6) | Chris (10) | Mary (6) | Jay (2)

◀ Prev Sub-step 11 / 12 Next ▶

```
if (compare→next == NULL && compare→pain ≥ pain) // NULL && 5≥2? → TRUE
```

Jack is the last node and 5 ≥ 2 → **Sub-case 3b-ii: append at end.**

```
$ Jay 2
```

ER Waiting Room — Pointer Trace Visualizer

Watch compare & compare2 traverse the singly-linked list

head · new node · compare · compare2 · existing

Init | Jack (5) | Jose (6) | Chris (10) | Mary (6) | Jay (2)

◀ Prev Sub-step 12 / 12 Next ▶

```
newnode→next = NULL;
compare→next = newnode; // Jack→Jay
```

Appended! Jay(2) is the new tail. Lowest pain = lowest priority = last in line.

```
$ Jay 2
$ Chris (10) → Jose (6) → Mary (6) → Jack (5) → Jay (2) → End of waiting room list for now!
```

7 mins

Alternatively, students can check
https://github.com/yeanavt/emergencyroomcheckin

FOLLOW-UP QUIZ 1: TRUE OR FALSE?

1. TO INSERT CHRIS WITH THE PAIN LEVEL 10, IT IS MOST COMPLEX SINCE IT BECOMES THE HEAD NODE RIGHT AWAY.

FOLLOW-UP QUIZ 1: TRUE OR FALSE?

1. TO INSERT CHRIS WITH THE PAIN LEVEL 10, IT IS MOST COMPLEX SINCE IT BECOMES THE HEAD NODE RIGHT AWAY.

FALSE

FOLLOW-UP QUIZ 2: TRUE OR FALSE?

2. THE CURRENT ER CHECK-IN SYSTEM DOES NOT KEEP THE ORDER OF A PATIENT'S TIME OF ARRIVAL DIRECTLY

FOLLOW-UP QUIZ 2: TRUE OR FALSE?

2. THE CURRENT ER CHECK-IN SYSTEM DOES NOT KEEP THE ORDER OF A PATIENT'S TIME OF ARRIVAL DIRECTLY

TRUE

FOLLOW-UP QUIZ 3: WHAT IS NOT IN STRUCT IN THE GIVEN CODE?

1. PATIENTINFO *NEXT
2. INT NURSE_ID
3. STRING FNAME
4. STRING LNAME
5. INT PAIN

FOLLOW-UP QUIZ 3: WHAT IS NOT IN STRUCT IN THE GIVEN CODE?

1. PATIENTINFO *NEXT
2. INT NURSE_ID
3. STRING FNAME
4. STRING LNAME
5. INT PAIN

FOLLOW-UP QUIZ 4: AGREE OR DISAGREE?

4. THE CURRENT SYSTEM IS SOMEWHAT UNFAIR AND INEFFICIENT.

FOLLOW-UP QUIZ 4: AGREE OR DISAGREE?

4. THE CURRENT SYSTEM IS UNFAIR BECAUSE IT TAKES THE PATIENT'S FIRST NAME AND SELF-EVALUATED PAIN LEVEL ONLY

DISCUSSION USING ETHICAL REASONING FOR IMPROVING THE GIVEN ER CHECK-IN SYSTEM (5 MINS)

GUIDING QUESTIONS TO ANSWER

1. IDENTIFY FLAWS
2. WHAT IS YOUR GROUP'S FOCUS TO ADDRESS THEM?
3. NEW FEATURES TO IMPROVE THE GIVEN CHECK-IN SYSTEM?
4. EXPECTED CHALLENGES TO OVERCOME THE FLAWS OR AGREEING TO THE SOLUTION?

LET’S SHARE WHAT YOU DISCUSSED!

– PART OF THE GROUP ASSIGNMENT

DATA STRUCTURE	FINDINGS / POTENTIAL IMPROVEMENT?	CHALLENGES?
Priority Queue	self-eval pain level can be inaccurate and abuse the system	
Singly connected linked lists	Can we sort the singly linked list using a sort algorithm?	
Struct	Takes only patient’s first name and self-evaluated pain level. One pointer to track the next node	

Group Assignments

1. Improved version of ER check-in system with comments
2. Group report (~2 pages including the discussion chart)

Q & A

Thank you!